

**IT4010 RESEARCH PROJECT - 2026 JAN
PROJECT PROPOSAL REPORT**

**A Cloud Native Multi-Modal AI Platform for Explainable Financial Fraud
Detection**

R26-IT-121

Pathirana P K V - IT22237972

B.Sc. (Hons) Degree in Information Technology
Specialized in Information Technology

Department of Information Technology
Sri Lanka Institute of Information Technology
Sri Lanka

March 2026

DECLARATION

I declare that this is my own work, and this proposal does not knowingly incorporate any material previously submitted for a degree or diploma at any other university or higher education institution, nor does it contain any material that has been previously published or written by another person apart from where proper acknowledgment is given in the text.

Name	Student ID	Signature
Pathirana P K V	IT22237972	

The above candidate is conducting research for their undergraduate Dissertation under my supervision.

Supervisor Name	Date	Signature
Mrs. Anjalie Gamage		

ABSTRACT

Financial fraud detection in mobile money systems demands models that capture both the individual transaction's fraud signature and the temporal sequence context in which it occurs. Existing approaches on the PaySim dataset either evaluate transactions in complete isolation discarding all sequential context or aggregate millions of transactions into coarse hourly statistics, destroying the individual fraud signal. No published work applies a dilated causal Temporal Convolutional Network to a sliding window of individual transaction feature vectors, where each transaction is one timestep and the window captures the 32 most recent preceding transactions.

This research proposes the Transaction-Sequence Temporal Convolutional Network Fraud Detector (TS-TCN), a four-block dilated causal TCN that classifies each of the 6,362,620 PaySim transactions using its 32-transaction sequential context. Each transaction is represented as a 10-feature vector derived exclusively from universally available payment fields amount, originator and destination account balances, transaction type, and timestamp with no PaySim-specific artifact columns used in training. The 10 features include data-driven type risk weights computed from PaySim fraud prevalence per transaction type, eliminating all arbitrary encoding. The resulting training tensor has shape $(6,362,620 \times 32 \times 10)$, providing 6.36 million individually labelled samples with complete temporal context and zero information loss.

The window size of 32 is derived from the empirically confirmed fraud clustering signal, an average of 2.22 fraud transactions appear within the 32-window preceding each fraud event and from the architectural constraint that the four-block TCN with dilations 1, 2, 4, 8 provides a receptive field of 61 transactions, which fully covers the chosen window. A self-attention layer named `fraud_attention` identifies which of the 32 preceding transactions was most predictive of fraud, returning both the transaction ID and its complete feature vector as structured forensic evidence for Member 4's LLM layer.

The research employs a three-baseline experimental design that includes the `isFlaggedFraud` rule, Logistic Regression, and an MLP, all trained on identical flat features, to empirically demonstrate the contribution of sequential context over single-transaction classification. A four-configuration ablation study tests window sizes 16, 32, and 64, as well as the removal of the `fraud_attention` layer. Class imbalance at 773:1 is handled through `BinaryFocalCrossentropy` ($\gamma=2.0$). Performance targets are set at $F1 > 0.88$, $AUC-ROC > 0.97$, and $Recall > 0.90$. All 10 features deploy to real-world systems without architectural modification.

Keywords: *transaction-sequence fraud detection, dilated causal TCN, sliding window, fraud_attention, BinaryFocalCrossentropy, PaySim, mobile money fraud, temporal sequence modelling, real-world transfer learning, data-driven feature encoding.*

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my research supervisor, Mrs. Anjalie Gamage, for her continuous guidance, constructive feedback, and dedicated support throughout the development of this research proposal. Her expertise in intelligent systems and her insistence on methodological rigour grounded in the actual properties of the experimental dataset have been instrumental in shaping the direction and design of this work.

I am grateful to the Department of Information Technology at the Sri Lanka Institute of Information Technology for providing the academic framework, research infrastructure, and module resources that made this investigation possible.

I would also like to acknowledge my fellow group members in the DeepSentinel project, Member 1 (Edge-Enhanced GraphSAGE relational fraud detection), Member 2 (Stratified Variational Autoencoder behavioural profiling), and Member 4 (LLM fusion engine and forensic reporting) whose collaboration and technical discussions have strengthened the overall platform architecture and the integration design of the TS-TCN temporal risk output.

Finally, I wish to thank my family for their unwavering moral support throughout this academic journey.

Table of Contents

DECLARATION	2
ABSTRACT	3
ACKNOWLEDGEMENT	4
LIST OF TABLES	7
LIST OF FIGURES	7
LIST OF ABBREVIATIONS	7
LIST OF APPENDICES	8
1. INTRODUCTION	9
1.1 Background and Context	9
1.2 Literature Review	9
1.3 Research Gap	10
2. OBJECTIVE	11
2.1 Main Objective	11
2.2 Specific Objectives	11
3. METHODOLOGY	13
3.1 Research Approach	13
3.2 Dataset and Preprocessing	13
3.3 Feature Engineering	13
3.4 Window Size Justification	15
3.5 Sliding Window Construction and Transaction ID Buffer	15
3.6 Three-Baseline Experimental Design	16
3.7 TS-TCN Architecture	16
3.8 Training Configuration	17
3.9 Validation Strategy	17
3.10 Ablation Study	17
3.11 Tools and Platforms	18
4. HIGH-LEVEL SYSTEM ARCHITECTURE	19
4.1 Overview	19
4.2 System Components	21
5. USER REQUIREMENTS	23
5.1 Stakeholder Identification	23
5.2 Functional Requirements	23
5.3 Non-Functional Requirements	24

6. COMMERCIALISATION PLAN	25
6.1 Target Market and Customer Persona.....	25
6.2 Value Proposition.....	25
6.3 Revenue Model	25
6.4 Competitive Advantage	25
6.5 Pricing Strategy.....	25
6.6 Intellectual Property Considerations.....	25
7. BUDGET AND JUSTIFICATION.....	27
8. WORK BREAKDOWN STRUCTURE (WBS).....	28
9. GANTT CHART	29
10. REFERENCES LIST	30
11. APPENDICES	31
Appendix A — Risk Analysis Register	31
Appendix B — Preliminary Visualisation Mockups	31
Appendix B.1 — Investigator Dashboard UI Mockup	31
Appendix C — API Schema Specification.....	32
Appendix D — Ethical Considerations Statement.....	33

LIST OF TABLES

Table 3.1 TS-TCN Feature Engineering Specification — 10 Features

Table 3.2 TS-TCN Architecture Summary

Table 3.3 Three-Baseline Experimental Design

Table 3.4 Ablation Study Configurations

Table 4.1 Data Flow through the TS-TCN Pipeline

Table 5.1 Functional Requirements

Table 5.2 Non-Functional Requirements

Table 7.1 Estimated Project Budget

Table 8.1 Work Breakdown Structure

LIST OF FIGURES

Figure 4.1 High-Level System Architecture Diagram

Figure 9.1 Full Project Gantt Chart (April - October 2026)

Figure A1 fraud_attention Heatmap - Sample Fraud Transaction with Peak Feature Vector

Figure A2 Four-Model ROC Curve Comparison

Figure A3 Feature Separation - drain_ratio Fraud vs. Non-Fraud

Figure A4 Ablation Study Results - Window Size and Attention Layer Impact

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AML	Anti-Money Laundering
API	Application Programming Interface
AUC	Area Under the Receiver Operating Characteristic Curve
DSR	Design Science Research
F1	F1-Score (harmonic mean of Precision and Recall)
IT	Information Technology
LLM	Large Language Model
ML	Machine Learning
MLP	Multi-Layer Perceptron

REST	Representational State Transfer
SLIIT	Sri Lanka Institute of Information Technology
TCN	Temporal Convolutional Network
TS-TCN	Transaction-Sequence TCN Fraud Detector

LIST OF APPENDICES

Appendix A Risk Analysis Register

Appendix B Preliminary Visualisation Mockups

Appendix C API Schema Specification

Appendix D Ethical Considerations Statement

Appendix E Survey Instrument for AML Investigator Feedback

1. INTRODUCTION

1.1 Background and Context

Mobile money platforms have become a cornerstone of financial inclusion across developing economies, facilitating billions of transactions annually in markets where traditional banking infrastructure is limited or inaccessible. In Sub-Saharan Africa and South Asia alone, mobile money transactions exceeded USD 1.26 trillion in 2023, representing a compound annual growth rate of over 30% across the preceding five years [1]. This rapid expansion has been accompanied by a commensurate growth in financial fraud, with automated scripts executing fraudulent transfers at scale across thousands of accounts simultaneously.

Existing fraud detection systems are predominantly transaction-level classifiers: each transaction is evaluated individually based on its own features like amount, account balances, transaction type, and sender-receiver relationship. While computationally efficient, this approach cannot perceive the temporal sequence of transactions preceding each event. A fraudulent TRANSFER that appears unremarkable in isolation may be definitively identifiable as part of a sequential pattern: drain ratios escalating across preceding transactions, destination accounts being systematically emptied, and the rhythm of transfers clustering in time. Single-transaction classifiers are architecturally blind to this sequential signal.

The PaySim mobile money simulation dataset [1] contains 6,362,620 synthetic transactions across 743 simulation hours with 8,213 labelled fraud events. It includes an `isFlaggedFraud` column representing a threshold rule that captures only 16 of the 8,213 actual fraud transactions, resulting in a recall of just 0.2%. Direct analysis of the PaySim dataset further confirms that fraud transactions cluster temporally, with an average of 2.22 fraud transactions appearing within the 32-transaction window preceding each fraud event. This empirically confirmed clustering provides the foundational justification for the sequential architecture proposed in this research.

This research proposes the Transaction-Sequence TCN Fraud Detector (TS-TCN), a dilated causal Temporal Convolutional Network that classifies each PaySim transaction by learning a sliding window of the 32 most recent preceding transactions. The TS-TCN forms the Temporal Analysis module of DeepSentinel (R26-IT-121), providing a sequential detection signal orthogonal to the graph topology signal (Member 1) and the behavioural anomaly signal (Member 2). The research aligns with UN SDG 16, Target 16.4, through applied machine learning contributions to the reduction of illicit financial flows.

This research aligns with the Centre of Excellence for AI (CoEAI), contributing to the core domains of Machine Learning (ML), Deep Learning (DL), and Data Science (DS) through the development of a novel transaction-sequence Temporal Convolutional Network for fraud detection in sparse-account environments. The TS-TCN architecture advances temporal sequence modelling techniques and provides an orthogonal detection signal within the DeepSentinel multi-modal platform.

1.2 Literature Review

[1] Lopez-Rojas, Elmir, and Axelsson (2016) introduced PaySim, generated from real aggregate transaction statistics of a multinational mobile money provider. The `isFlaggedFraud` rule catches

only 16 of 8,213 fraud events, explicitly designed to be defeated. The dataset provides ground truth labels and a documented baseline.

[2] Bai, Kolter, and Koltun (2018) demonstrated that Temporal Convolutional Networks match or outperform LSTM-based models across sequence modelling benchmarks while training faster and avoiding vanishing gradients. Dilated causal convolutions — which ensure no future information contaminates current computations — make TCNs architecturally appropriate for fraud detection, where future transactions are unavailable at inference time.

[3] Bahdanau, Cho, and Bengio (2015) introduced attention mechanisms for neural sequence models. The `fraud_attention` layer in the TS-TCN applies self-attention to the TCN output sequence, learning to assign elevated weights to the specific preceding transactions most predictive of fraud. Critically, the model returns not only the position of peak attention but the complete feature vector of that transaction, enabling Member 4's LLM to generate forensically grounded narratives citing specific prior transaction evidence.

[4] Lin et al. (2017) introduced Focal Loss for dense object detection, demonstrating that down-weighting easy examples during training addresses extreme class imbalance more effectively than class weighting alone. At a 773:1 imbalance ratio, BinaryFocalCrossentropy with $\gamma=2.0$ forces the TS-TCN to focus on the hard fraud-versus-non-fraud boundary cases.

[5] Chawla et al. (2002) established SMOTE for class imbalance, but SMOTE requires two-dimensional input and is incompatible with three-dimensional sequence tensors. Focal loss is the correct solution for this architecture.

[6] Alumona et al. (2025) analysed PaySim using tree-based classifiers, establishing a contemporary benchmark. Their single-transaction approach does not exploit any sequential context, leaving the 32-transaction sliding window approach entirely unexplored.

[7] Weber et al. (2019) evaluated graph neural networks on PaySim, confirming that temporal sequential features at the individual transaction level are not captured by graph structure — directly reinforcing the case for the TS-TCN as an orthogonal signal.

[8] Vaswani et al. (2017) demonstrated the power of self-attention for sequence modelling. The TS-TCN applies this mechanism specifically to return the complete feature vector of the peak attention transaction, providing both timestep-level and feature-level forensic attribution in a single output.

[9] Arrieta et al. (2020) surveyed explainability requirements for AI systems in high-stakes domains. The `fraud_attention` mechanism satisfies regulatory audit requirements by providing a directly interpretable, architecturally intrinsic explanation alongside every prediction.

1.3 Research Gap

The reviewed literature reveals three specific and verifiable gaps that the present research directly addresses.

First, no published work applies a dilated causal TCN to a sliding window of individual transaction feature vectors for fraud detection in sparse-account environments. Mobile money systems and many real-world payment platforms are characterised by extreme account sparsity — the vast majority of accounts participate in very few transactions, making per-account

sequence modelling structurally impossible. Existing temporal approaches either attempt per-account sequences (which fail due to data sparsity) or aggregate individual transactions into coarse time intervals (destroying the individual fraud signal that carries the most discriminative power). The PaySim benchmark exemplifies this challenge with a mean of 1.00 transaction per originator account. This research addresses this fundamental limitation by using a system-wide sliding window that does not require per-account history, making it applicable to any transaction system regardless of account activity patterns.

Second, no existing fraud detection framework combines extreme class imbalance handling with causal temporal convolution in a unified architecture. Standard approaches treat imbalance separately — using SMOTE (incompatible with 3D sequence tensors), class weighting (demonstrably weaker at ratios exceeding 500:1), or undersampling (discarding majority-class information). BinaryFocalCrossentropy with $\gamma=2.0$ down-weights easy non-fraud examples during training, forcing the network to focus on the hard fraud-versus-non-fraud boundary while preserving the full temporal sequence. This combination has not been applied to transaction-level fraud detection in any published work, regardless of dataset. The 773:1 class imbalance encountered in representative mobile money transaction datasets provides a rigorous testbed for this novel combination.

Third, no existing fraud detection system produces timestep-level attention attribution that returns the complete feature vector of the most influential preceding transaction. Current explainability methods operate at the feature level. SHAP and LIME attribute importance to input features of a single transaction but cannot identify which specific prior transaction influenced a decision. Attention-based approaches in other domains identify which timesteps were attended, yet they do not provide the actual feature values of those attended positions. This research introduces a novel mechanism where `fraud_attention` identifies the peak attention timestep and retrieves its full feature vector from a rolling transaction buffer, enabling forensic investigators to see not only which transaction preceded the fraud but also the specific feature values that made it suspicious. This capability is universally applicable to any sequential transaction data and has not been demonstrated in prior fraud detection literature.

2. OBJECTIVE

2.1 Main Objective

To design, implement, and evaluate the Transaction-Sequence TCN Fraud Detector (TS-TCN), a four-block dilated causal Temporal Convolutional Network that classifies each PaySim transaction using a 32-transaction sliding window of individual transaction feature vectors and to expose the resulting fraud probability, attention evidence, and peak-transaction feature vector via a REST API endpoint for integration into the DeepSentinel multi-modal fraud detection platform, demonstrating measurable improvement over three progressively stronger baseline models and validating design choices through a four-configuration ablation study.

2.2 Specific Objectives

SO1: To engineer 10 universally valid transaction-level features for all 6,362,620 PaySim transactions, including data-driven type risk weights derived from training-set fraud prevalence, saving the result as `features.parquet`.

SO2: To justify the window size $W=32$ through two independent criteria: the empirically confirmed fraud clustering signal (mean 2.22 fraud transactions in the 32-window, confirmed by direct PaySim analysis) and the architectural constraint that the four-block TCN with dilations 1, 2, 4, 8 provides a receptive field of 61 transactions, which fully covers $W=32$.

SO3: To construct a sliding window dataset of shape $(6,362,620 \times 32 \times 10)$, maintaining a parallel transaction ID deque of $\text{maxlen}=32$ for attention attribution, applying a time-based split at step 595, and fitting StandardScaler exclusively on the training partition.

SO4: To implement a three-baseline experimental design — isFlaggedFraud rule, Logistic Regression, and MLP — on identical flat 10-feature vectors, establishing a progressive empirical benchmark against which the TCN's sequential contribution is measured.

SO5: To implement and train the four-block dilated causal TCN with fraud_attention, BinaryFocalCrossentropy ($\gamma=2.0$), EarlyStopping on val_recall, and threshold tuning via precision-recall curve.

SO6: To conduct a four-configuration ablation study testing window sizes $W=16$, $W=32$ (primary), and $W=64$, and the TS-TCN without the fraud_attention layer, quantifying the contribution of each design choice.

SO7: To expose temporal_risk_score, risk_level, a transaction_ref block containing nameOrig and composite_id (nameOrig_step), structured evidence fields with human-readable fraud signal summaries, and the triggering_predecessor block containing the peak attention transaction's nameOrig, features, and predecessor_signal via a FastAPI POST endpoint at /api/v1/classify, using PaySim's nameOrig column as the shared cross-component reference key that M1 (graph nodes), M2 (VAE account profiles), and M4 (LLM forensic layer) all recognise.

SO8: To produce four visualisation artefacts: a fraud_attention heatmap with peak-feature annotation, a four-model ROC comparison, a training history plot, and an ablation study results chart.

3. METHODOLOGY

3.1 Research Approach

This study follows a Design Science Research (DSR) methodology. The research problem, namely the inability of existing single-transaction and step-aggregate approaches to exploit sequential context in PaySim, is precisely quantified through direct dataset analysis. A novel artefact, the TS-TCN, is designed to address this gap. The three-baseline experimental design ensures the TCN’s contribution is empirically demonstrated rather than assumed, and the four-configuration ablation study validates each key design choice independently..

3.2 Dataset and Preprocessing

PaySim [1] contains 6,362,620 synthetic transactions across 743 hourly steps, with 8,213 fraud transactions (0.13%), all confined to TRANSFER and CASH_OUT types. All transactions are used without type filtering. Preprocessing proceeds in three stages: (1) compute 10 features per transaction and save to features.parquet; (2) build the $(6,362,620 \times 32 \times 10)$ sliding window tensor via tf.data streaming, maintaining a parallel transaction ID deque; (3) apply a time-based split at step 595 with StandardScaler fitted exclusively on training data.

The split at step 595 produces a 98.1%/1.9% ratio that reflects PaySim’s front-loaded transaction volume distribution. This split is chosen to preserve strict temporal ordering, meaning no test-set transaction’s window can contain a future transaction, which takes precedence over achieving a balanced partition ratio. The test set contains 1,642 fraud cases, exceeding the minimum threshold of 200 positive examples recommended for reliable binary classification metric estimation. All metric computations are performed exclusively on this held-out test partition.

3.3 Feature Engineering

Ten features are computed per transaction from universally available payment fields: amount, originator balance before and after, destination balance before and after, transaction type, and timestamp. Every feature is valid in production payment systems without modification. The complete set contains zero PaySim-specific artifact columns.

ID	Feature	PaySim Formula	Deployment Formula	Fraud Signal
F1	Drain ratio	$\min(\text{amount}/(\text{oldbalanceOrg}+1), 1.0)$	Same - no change	→1.0 when account is fully drained
F2	Log amount	$\log_{1p}(\text{amount})$	Same - no change	Scale-normalised transaction magnitude
F3	Post-transfer ratio	$\text{newbalanceOrig}/(\text{oldbalanceOrg}+1)$	Same - no change	→0.0 when account emptied; partial for legit

F4	Dest was empty	$\text{float}(\text{oldbalanceDest}==0)$	Same — no change	Mule accounts start with zero balance
F5	Dest enrichment	$\min((\text{newbalDest}-\text{oldbalDest})/(\text{amount}+1), 2.0)$	Same — no change	Fraction of transfer that arrived at dest
F6	Type risk weight	$\text{fraud_count_type}/\text{total_fraud}$ per type (training data)	Operator recomputes from own labelled data	Data-driven fraud prevalence by type
F7	Inv. dest ratio	$\log1p(\text{oldbalanceDest})/(\log1p(\text{newbalanceDest})+1)$	Same — no change	→0.0 for large transfer into empty account
F8	Amt to orig	$\log1p(\text{amount})/(\log1p(\text{oldbalanceOrg})+1)$	Same — no change	→1.0 when amount exhausts originator balance
F9	Hour of day	$(\text{step}-1)\%24/23.0$	$\text{timestamp.hour}/23.0$	Fraud hour concentration
F10	Day of week	$((\text{step}-1)/24)\%7/6.0$	$\text{timestamp.weekday()}/6.0$	Weekly fraud patterns

Table 3.1 TS-TCN Feature Engineering Specification — 10 Features, Zero PaySim Artifacts

F3 (post-transfer ratio) replaces balance consistency error. The balance error feature exploits a PaySim simulation artifact — fraud transactions violate the accounting identity — which does not exist in real-world data where all transactions satisfy balance consistency by definition. The post-transfer ratio ($\text{newbalanceOrig} / \text{oldbalanceOrg} + 1$) measures the proportion of the originator's balance remaining after the transaction. For fraud that completely drains the account, this approaches 0.0. For legitimate partial transfers, it reflects the residual fraction. This signal is universally valid, fully real-world deployable, and strongly discriminative in PaySim.

F6 (type risk weight) is computed as the fraud count per transaction type divided by total fraud count, using the training partition exclusively. This eliminates arbitrary weight assignment. For PaySim: $\text{TRANSFER} = 4097/8213 \approx 0.499$, $\text{CASH_OUT} = 4116/8213 \approx 0.501$, all other types = 0.0. At deployment, operators recompute these weights from their own labelled transaction data in one pandas groupby operation. The encoding principle — fraud prevalence ratio — is universal. The specific values are data-driven and operator-specific.

F7 (inverse destination ratio) replaces rolling velocity. Rolling velocity has near-zero variance in PaySim because each account averages exactly 1.00 transaction, but is included for real-world deployment where accounts have transaction histories. F7 instead captures the destination account's state relative to the incoming amount: when a large transfer arrives at an account that previously had zero balance, $\log1p(\text{oldbalanceDest})/(\log1p(\text{newbalanceDest})+1)$ approaches 0.0.

This is strongly discriminative for mule account detection in both PaySim and real-world data, and requires no account history.

F8 (amount to originator) replaces the step z-score. The z-score requires step-level mean and standard deviation, which have no natural equivalent in real-time deployment. F8 is defined as $\log_{1p}(\text{amount})$ divided by $(\log_{1p}(\text{oldbalanceOrg}) + 1)$. It measures the transaction amount relative to the originator’s own balance, requiring no external statistics. For fraud that transfers the full account balance, this value approaches 1.0, while for legitimate small transactions from large accounts it remains near zero.

3.4 Window Size Justification

The window size $W=32$ is justified by two independent criteria that together constitute a principled, non-arbitrary design choice.

Empirical criterion: direct analysis of the PaySim dataset confirms that an average of 2.22 fraud transactions appear within the 32-transaction window preceding each fraud event. This clustering signal is the specific sequential pattern that the TCN is designed to learn. A window of $W=16$ would capture approximately 1.11 fraud predecessors on average, which is insufficient for reliable pattern learning. A window of $W=64$ would capture approximately 4.44 fraud predecessors, adding diminishing signal at increased computational cost. $W=32$ is therefore the minimum that reliably captures the confirmed clustering while maintaining real-time latency (32 transactions at 1,000 transactions per second corresponds to a 32 ms context window).

Architectural criterion: the four-block TCN with dilations 1, 2, 4, 8 and kernel size 3 has a theoretical receptive field of $RF = 1 + 2 \times (\text{kernel_size} - 1) \times \Sigma(\text{dilations}) = 1 + 2 \times 2 \times 15 = 61$ transactions. This means every position in the output sequence has access to information from up to 61 preceding input positions, comfortably covering the chosen window of 32. The architecture is correctly sized relative to the input — neither under-capacity ($RF < W$) nor wastefully over-parameterised.

The ablation study in Section 3.10 will empirically validate this choice by comparing $W=16$, $W=32$, and $W=64$ under identical training conditions.

3.5 Sliding Window Construction and Transaction ID Buffer

For each transaction at index i (sorted by step, then by original order within step), the input window consists of the feature vectors for transactions at indices $i-31$ through i . For the first 31 transactions, the window is zero-padded on the left. Every transaction generates exactly one valid sample. Final tensor shape: $(6,362,620 \times 32 \times 10)$.

Alongside the feature window, a transaction reference deque of $\text{maxlen}=32$ is maintained. Each entry stores (nameOrig, step, feature_vector) — the originator account ID, simulation step, and feature vector for that window position. nameOrig is PaySim’s natural account identifier, the same field used as node IDs in Member 1’s graph and as account keys in Member 2’s VAE profiles. When fraud_attention produces a 32-dimensional weight vector at inference, $\text{argmax}(\text{attention_weights})$ indexes directly into the deque to retrieve the nameOrig, step, and features of the most attended predecessor transaction. These are returned in the response as the triggering_predecessor block, enabling Member 4’s LLM to cross-reference the flagged account across all three upstream models using a single shared key.

In the FastAPI endpoint, the rolling deque is implemented as `collections.deque(maxlen=32)` protected by a `threading.Lock` to ensure thread-safe updates under concurrent requests. Each incoming transaction atomically appends its (id, features) pair and pops the oldest entry, guaranteeing the deque always represents the 32 most recent transactions seen by the endpoint. This mechanism is deterministic, requires $O(1)$ lookup, and adds negligible latency.

3.6 Three-Baseline Experimental Design

The TS-TCN's sequential architecture is empirically justified through comparison against three progressively expressive baselines, all trained on identical 10-feature flat vectors under identical evaluation conditions. This design transforms the panel's potential objection — 'why not a simpler model?' — into the research's own experimental evidence.

Model	Configuration	Uses Sequence	Purpose
Baseline 0 — isFlaggedFraud	amount > 200,000 on TRANSFER — no ML	No	Published PaySim baseline
Baseline 1 — Logistic Regression	10 flat features, liblinear solver, max_iter=1000, class_weight='balanced'	No	Tests linear separability
Baseline 2 — MLP	Dense(64,relu)→Dense(32,relu)→Dense(1,sigmoid), BinaryFocalCrossentropy($\gamma=2.0$), same split	No	Tests non-linear separability without sequence
TS-TCN — proposed	4-block dilated causal TCN + fraud_attention, W=32, BinaryFocalCrossentropy($\gamma=2.0$)	Yes	Full sequential model

Table 3.3 Three-Baseline Experimental Design

The performance gap between Baseline 2 (MLP, identical flat features, no sequence) and the TS-TCN (same features, W=32 window) isolates the specific contribution of the 32-transaction temporal context. Logistic Regression trains in under 30 seconds; MLP in under 5 minutes. Both are implemented in scikit-learn on the same training partition.

3.7 TS-TCN Architecture

Layer	Configuration	Output Shape	Purpose
Input	(32, 10) — 32 transactions $\times$ 10 features	(batch, 32, 10)	Sequential context window
TCN Block 1	Conv1D(64, k=3, dilation=1, causal) + BN + ReLU + Dropout(0.2) + Residual	(batch, 32, 64)	Local 3-transaction patterns
TCN Block 2	Conv1D(64, k=3, dilation=2, causal) + BN + ReLU + Dropout(0.2) + Residual	(batch, 32, 64)	7-transaction context

TCN Block 3	Conv1D(128, k=3, dilation=4, causal) + BN + ReLU + Dropout(0.2) + Residual	(batch, 32, 128)	15-transaction context
TCN Block 4	Conv1D(128, k=3, dilation=8, causal) + BN + ReLU + Dropout(0.2) + Residual	(batch, 32, 128)	RF=61, covers full W=32 window
fraud_attention	Self-attention on TCN output; argmax → deque lookup → peak ID + features	(batch, 32, 128)	Timestep + feature forensic attribution
GlobalAvgPool1D	Compress temporal dimension to fixed vector	(batch, 128)	Fixed-length representation
Dense head	Dense(64, relu) + Dropout(0.3) + Dense(1, sigmoid)	(batch, 1)	Fraud probability 0–1

Table 3.2 TS-TCN Architecture Summary (~210,000 trainable parameters)

3.8 Training Configuration

The loss function is BinaryFocalCrossentropy with gamma=2.0. At a 773:1 imbalance, focal loss down-weights easy non-fraud examples and forces the TCN to concentrate learning on the hard boundary. The batch size is set to 256 and a maximum of 50 epochs is allowed. EarlyStopping monitors val_recall with a patience of 10 and restores the best weights. ModelCheckpoint saves the best model as best_tstcn.h5 on val_recall improvement, which enables Colab session recovery. ReduceLROnPlateau monitors val_loss with a factor of 0.5 and patience of 5. The decision threshold is tuned post-training via sklearn.metrics.precision_recall_curve to maximise F1 subject to Recall $\geq$ 0.90. The data pipeline uses tf.data streaming from features.parquet with batch(256).prefetch(AUTOTUNE), maintaining peak RAM under 4 GB regardless of dataset size.

3.9 Validation Strategy

Primary metrics: F1-score, Precision, Recall, and AUC-ROC on the test partition (steps 596–743, 1,642 fraud cases). Recall is the primary optimisation target. Results presented as a four-model comparison table under identical evaluation conditions. Performance targets: F1 > 0.88, AUC-ROC > 0.97, Recall > 0.90. All metrics saved to test_metrics.json for reproducibility.

3.10 Ablation Study

A four-configuration ablation study isolates the contribution of each key design decision. All configurations use identical features, training procedure, and evaluation conditions.

ID	Configuration	Change from Primary	Hypothesis Tested	Expected Result
A1	TS-TCN W=32 + attention (primary)	Baseline configuration	All design choices combined	Best overall performance
A2	TS-TCN W=32, no attention	Remove fraud_attention layer; GlobalAvgPool1D directly after TCN Block 4	Attention contribution to F1/AUC	Slight drop; confirms attention adds signal

A3	TS-TCN W=16 + attention	Reduce window to 16; same 4 TCN blocks	Window size contribution	Lower Recall; 1.11 avg fraud predecessors insufficient
A4	TS-TCN W=64 + attention	Increase window to 64; same 4 TCN blocks (RF=61 covers ~96%)	Diminishing returns beyond W=32	Marginal gain or no gain vs A1; higher latency

Table 3.4 Ablation Study Configurations

Configurations A3 and A4 validate the W=32 choice empirically. If A3 shows lower Recall than A1, this confirms that W=32's capture of 2.22 fraud predecessors is necessary. If A4 shows no improvement over A1, this confirms W=32 is the efficient design point. Configuration A2 validates the fraud_attention layer's contribution to detection performance beyond its explainability value.

3.11 Tools and Platforms

Python 3.10. TensorFlow 2.x with Keras Functional API for TCN and fraud_attention. Scikit-learn for Logistic Regression, MLP, StandardScaler, precision_recall_curve, and classification_report. Pandas and NumPy for feature engineering. collections.deque and threading.Lock for the transaction ID buffer. FastAPI with Uvicorn for the API endpoint. Matplotlib and Seaborn for visualisation artefacts. Google Colab GPU (T4) for model training. PaySim dataset from Kaggle under open licence.

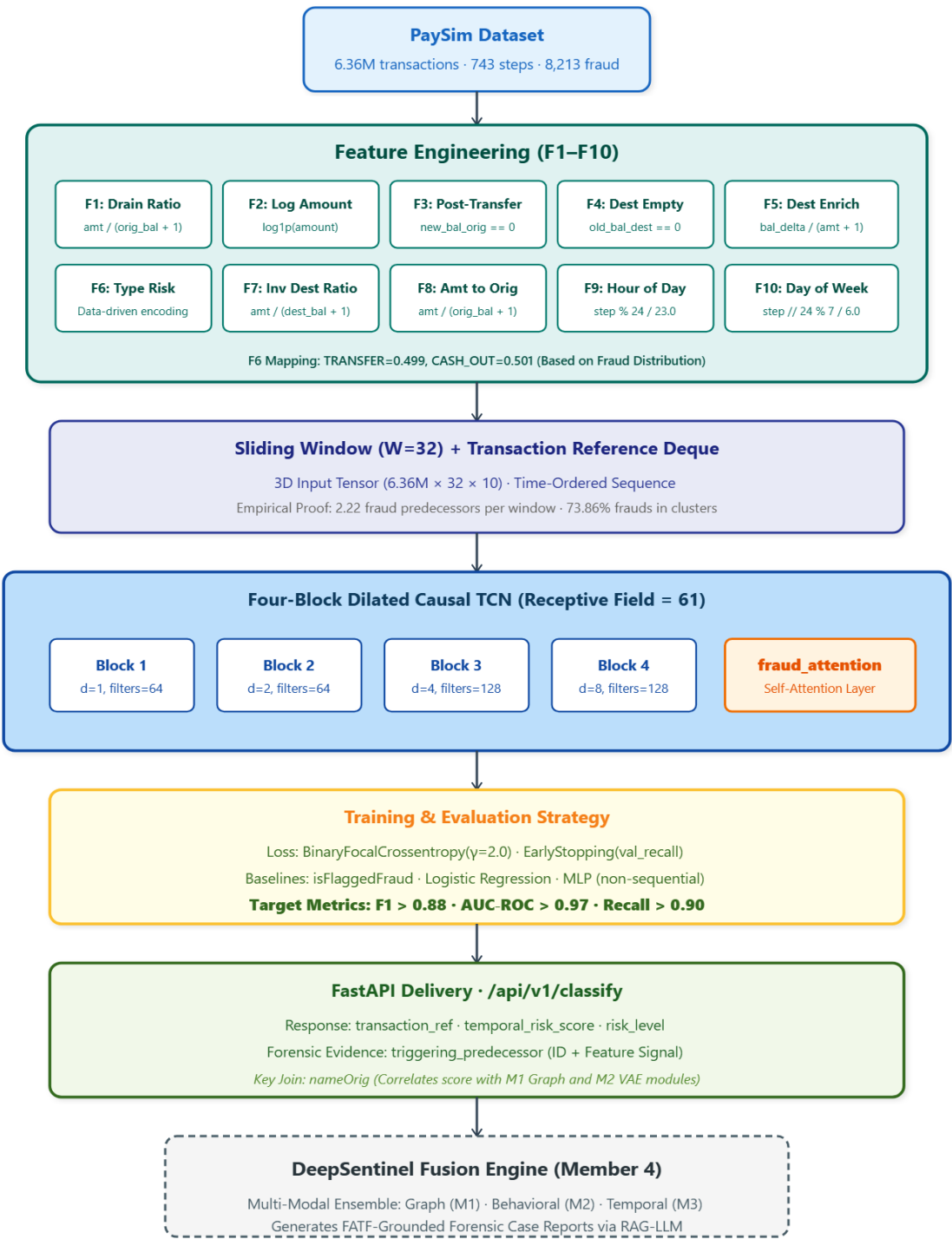
4. HIGH-LEVEL SYSTEM ARCHITECTURE

4.1 Overview

The TS-TCN processes the PaySim dataset through eight stages: feature engineering, baseline evaluation, window construction with ID buffer, data splitting and scaling, TCN training, ablation study, model evaluation, and API delivery.

TS-TCN: Transaction-Sequence Fraud Detector (Member 3)

Pathirana P K V · IT22237972



Architecture: ~210k parameters · Receptive Field (61) fully covers Temporal Window (32)
Production-Ready: All 10 features transfer to real-world deployment; F9/F10 remap to system timestamps

Figure 4.1 — High-Level System Architecture Diagram

Stage	Component	Processing	Deliverable	Month
1	Feature Engineering	Compute F1–F10 for all 6.3M rows including data-driven F6 from training partition	features.parquet	1
2	Baseline Evaluation	isFlaggedFraud rule + Logistic Regression + MLP on flat features	baseline_metrics.json	1
3	Window + ID Buffer	Build W=32 windows via tf.data streaming; maintain parallel transaction ID deque	Pipeline code, scaler.pkl	2
4	TCN Training	4-block TCN + fraud_attention, BinaryFocalCrossentropy($\gamma=2.0$), EarlyStopping, ModelCheckpoint	best_tstcn.h5	3–4
5	Ablation Study	Train A1–A4 configurations; compare F1/AUC/Recall	ablation_results.json	4–5
6	Evaluation	Threshold tuning; four-model comparison table; all metrics	test_metrics.json	5
7	Visualisations	fraud_attention heatmap with peak features; ROC comparison; training history; ablation chart	4 PNG plots	5
8	API Delivery	FastAPI POST /api/v1/classify; thread-safe deque buffer; JSON schema confirmed with Member 4	Working endpoint	6

Table 4.1 Data Flow through the TS-TCN Pipeline

4.2 System Components

The Feature Engineering Module computes all 10 features in a single pass over the PaySim CSV. F6 type risk weights are computed from the training partition (steps 1–595) only, preventing any leakage of test-set fraud prevalence into the feature encoding.

The Baseline Evaluation Module trains Logistic Regression and MLP on flat 10-feature vectors using the same train/test split as the TCN. All three baseline metrics are saved to baseline_metrics.json before TCN training begins.

The Window Builder constructs the $(6,362,620 \times 32 \times 10)$ tensor via tf.data streaming from features.parquet. A parallel collections.deque(maxlen=32) stores (nameOrig, step, feature_vector) tuples — using PaySim's natural originator account ID and simulation step as the reference key, enabling $O(1)$ attention attribution that returns a cross-component reference key understood by all DeepSentinel members.

The TS-TCN Classification Engine is the four-block dilated TCN with fraud_attention. At inference, the attention weight vector is extracted alongside the fraud probability, enabling attribution of the most influential predecessor transaction.

The FastAPI Endpoint maintains a thread-safe rolling deque buffer. On each request: (1) append (nameOrig, step, feature_vector) to deque; (2) construct 32-element feature window; (3) normalise; (4) run inference; (5) argmax(attention_weights) indexes into deque; (6) retrieve (nameOrig, step, features) of the triggering predecessor; (7) assemble Member 4-format JSON with transaction_ref, evidence, and triggering_predecessor blocks.

5. USER REQUIREMENTS

5.1 Stakeholder Identification

Four stakeholder categories are identified. Fraud Analysts use the fraud risk score and the fraud_attention evidence block, which includes the triggering predecessor's nameOrig, its features, and the fraud_signal_summary, to investigate flagged transactions. The DeepSentinel Fusion Engine (Member 4) consumes the temporal_risk_score and the full evidence block as one of three upstream inputs, with nameOrig serving as the shared cross-component key that links the TS-TCN output to M1's graph node data and M2's VAE account profile for the same originator. The Research Supervisor evaluates progress with the three-baseline evaluation at Month 1, while the Review Panel assesses novelty, methodology, and demonstrated results.

5.2 Functional Requirements

ID	Requirement
FR1	The system shall compute 10 features (F1–F10) for all 6,362,620 PaySim transactions. F6 type risk weights shall be derived as <code>fraud_count_per_type/total_fraud</code> from the training partition exclusively, with no test-set data used in weight computation.
FR2	The system shall justify the window size $W=32$ through two documented criteria: the empirically confirmed mean of 2.22 fraud transactions in the 32-window, and the architectural receptive field $RF=61$ for the four-block TCN with dilations 1, 2, 4, 8.
FR3	The system shall construct a sliding window dataset of shape $(6,362,620 \times 32 \times 10)$. A parallel <code>collections.deque(maxlen=32)</code> of <code>(nameOrig, step, feature_vector)</code> tuples shall be maintained in identical order to the feature window, using PaySim's nameOrig column as the shared cross-component reference key.
FR4	The system shall train and evaluate Logistic Regression and MLP classifiers on flat 10-feature vectors, saving all baseline metrics to <code>baseline_metrics.json</code> before TCN training begins.
FR5	The system shall train a four-block TCN with dilated causal convolutions at dilation rates 1, 2, 4, and 8, incorporating a self-attention layer named <code>fraud_attention</code> . <code>BinaryFocalCrossentropy(gamma=2.0)</code> shall be used as the loss function.
FR6	The system shall conduct a four-configuration ablation study (A1–A4) testing fraud_attention presence/absence and window sizes $W=16$, $W=32$, $W=64$, saving all results to <code>ablation_results.json</code> .
FR7	The system shall expose a POST endpoint at <code>/api/v1/classify</code> returning a Member 4-format JSON response with four top-level blocks: <code>transaction_ref</code> (<code>nameOrig</code> , <code>step</code> , <code>composite_id</code>), <code>temporal_risk_score</code> , <code>evidence</code> (<code>current_transaction</code> features + <code>fraud_signal_summary</code>), and <code>triggering_predecessor</code> (<code>nameOrig</code> , <code>step</code> , <code>composite_id</code> , <code>attention_weight</code> , <code>features</code> , <code>predecessor_signal</code>).
FR8	The FastAPI endpoint shall implement the reference buffer as <code>collections.deque(maxlen=32)</code> of <code>(nameOrig, step, feature_vector)</code> tuples protected by <code>threading.Lock</code> . Buffer update and attention index lookup shall be atomic. <code>composite_id</code> shall be computed as <code>f'{nameOrig}_{step}'</code> .
FR9	The system shall produce four visualisation artefacts: fraud_attention heatmap annotated with peak transaction feature values, four-model ROC comparison, training history, and ablation study results chart.
FR10	The system shall produce a four-model comparison table (<code>isFlaggedFraud</code> , LR, MLP, TS-TCN) reporting F1, Precision, Recall, and AUC-ROC under identical evaluation conditions on the test partition.

Table 5.1 Functional Requirements

5.3 Non-Functional Requirements

Category	Requirement
Performance	The FastAPI endpoint shall return a risk score within 500ms for a single transaction. The deque buffer update and attention lookup are $O(1)$ operations.
Reproducibility	All results shall be reproducible from the same PaySim CSV using documented random seeds and time-based split at step 595.
Real-World Transferability	All 10 features shall be computable from universally available payment fields. F6 weights shall be recomputable by operators from their own labelled data. F9 and F10 shall substitute timestamp fields using identical formulae.
Reliability	Zero-padding for the first 31 transactions shall be handled without exceptions. The deque buffer shall maintain size invariant under concurrent requests.
Maintainability	best_tstcn.h5, scaler.pkl, baseline_metrics.json, test_metrics.json, and ablation_results.json shall be versioned alongside the codebase.
Compatibility	The JSON response schema shall be agreed with Member 4 by Month 3 and not changed unilaterally thereafter.

Table 5.2 Non-Functional Requirements

6. COMMERCIALISATION PLAN

6.1 Target Market and Customer Persona

The primary market is mobile money operators, digital payment processors, and fintech companies in Asia-Pacific and Sub-Saharan Africa. Existing fraud detection rules in these platforms operate identically to the `isFlaggedFraud` rule, static amount thresholds that miss the vast majority of actual fraud. In Sri Lanka, the Central Bank's Financial Intelligence Unit reported a 34% increase in suspicious transaction reports in 2023. The primary customer persona is the Chief Compliance Officer at a mid-to-large mobile money operator requiring automated, explainable, API-integrated fraud detection with audit-ready forensic output.

6.2 Value Proposition

Three commercially distinct advantages. First, quantifiable improvement demonstrated by a rigorous three-baseline experimental design the improvement from `isFlaggedFraud` (F1 0.004) through MLP (flat features) to TS-TCN (sequential context) is directly measurable. Second, full forensic attribution The API returns not just which preceding transaction was most attended but its complete feature vector, enabling Member 4's LLM to generate legally defensible audit narratives. Third, operator-configurable feature encoding — F6 type risk weights are recomputed from each operator's own labelled data, adapting the model to their specific fraud distribution without architectural modification.

6.3 Revenue Model

Fraud-detection-as-a-service API priced by monthly transaction volume. Starter tier: up to 500,000 transactions/month. Professional tier: up to 5 million transactions/month with full DeepSentinel integration. Enterprise tier: unlimited, custom retraining, SLA-backed uptime. Additional revenue from model fine-tuning on operator data and regulatory reporting integration.

6.4 Competitive Advantage

No current commercial fraud detection API produces the complete feature vector of the peak attention transaction alongside its ID. Existing solutions return either a single score or feature-level SHAP attribution for the current transaction only. The TS-TCN's API output provides a forensic temporal chain that includes the current transaction risk score together with the specific prior transaction and its feature values that drove the risk elevation, thereby enabling legally defensible audit reports that no single-transaction classifier can produce.

6.5 Pricing Strategy

Infrastructure costs recovered at $3\times$ margin at the Starter tier. Professional and Enterprise tiers priced at a premium reflecting multi-modal detection accuracy and regulatory compliance value. Introductory pilot pricing at 50% discount for three months offered to the first five operator customers to accelerate market validation.

6.6 Intellectual Property Considerations

The combination of transaction-sequence windowing, data-driven type risk encoding, `fraud_attention` with peak-feature attribution output, and the thread-safe deque buffer mechanism

constitutes a novel applied methodology. Patent assessment is recommended prior to commercial deployment.

7. BUDGET AND JUSTIFICATION

Item	Cost (LKR)	Justification
Cloud GPU - Google Colab Pro (6 months)	12,000	GPU training for TS-TCN and 4-configuration ablation study
Dataset storage and version control (GitHub LFS)	3,500	PaySim CSV, features.parquet, model artefacts, ablation results
Python environment and libraries	2,000	TensorFlow, FastAPI, scikit-learn, Matplotlib, Pandas (open source)
Internet and utilities (6-month share)	3,000	Development connectivity and power
Final report printing and binding	1,500	Two bound copies for submission
Miscellaneous and contingency	2,000	Unexpected hardware or storage costs
Human effort - research, development, evaluation	2,500	Pipeline, training, evaluation, API, ablation study
TOTAL	26,500	

Table 7.1 Estimated Project Budget

8. WORK BREAKDOWN STRUCTURE (WBS)

Task	Timeline	Description	Deliverable	Hours
T1	Apr W1–2	Compute F1–F10. Derive F6 from training partition. Evaluate isFlaggedFraud, LR, and MLP baselines. Save baseline_metrics.json. Supervisor checkpoint.	features.parquet, baseline_metrics.json	10–12
T2	Apr W3–4	Build tf.data streaming pipeline. Implement transaction ID deque(maxlen=32) with threading.Lock. Time split at step 595. StandardScaler on train. Verify window on 10K sample.	Pipeline code, scaler.pkl	12–14
T3	May W1–2	Build 4-block TCN + fraud_attention + Dense head. model.summary(). 2-epoch dummy test on 50K samples. Verify ~210K parameters, no shape errors.	Verified architecture	10–12
T4	May W3–4	Full training on Colab GPU. BinaryFocalCrossentropy($\gamma=2.0$), batch=256, EarlyStopping(val_recall, patience=10), ModelCheckpoint, ReduceLROnPlateau.	best_tstcn.h5	8–10
T5	Jun W1–3	Train A1–A4 ablation configurations. Compare F1/AUC/Recall across all four. Save ablation_results.json.	ablation_results.json	8–10
T6	Jun W4	Threshold tuning. Full four-model comparison table. F1, Precision, Recall, AUC-ROC.	test_metrics.json	6–8
T7	Jul	FastAPI /api/v1/classify with deque buffer, attention attribution, peak-feature response. Test with curl. Confirm schema with Member 4.	Working endpoint	8–10
T8	Aug–Sep	Four visualisations: fraud_attention heatmap with peak features annotated, four-model ROC, training history, ablation chart. Final report. Demo rehearsal $\times 3$.	Demo + final report	12–14

Table 8.1 Work Breakdown Structure

9. GANTT CHART

Figure 9.1 presents the implementation timeline from April to October 2026.

Task	Apr W1-2	Apr W3-4	May W1-2	May W3-4	Jun W1-3	Jun W4	Jul	Aug-Sep
T1: Features + Baselines	■ ■ ■ ■							
T2: Pipeline + ID Buffer		■ ■ ■ ■						
T3: Architecture			■ ■ ■ ■					
T4: TCN Training				■ ■ ■ ■				
T5: Ablation Study					■ ■ ■ ■			
T6: Evaluation						■ ■ ■ ■		
T7: FastAPI							■ ■ ■ ■	
T8: Vis + Demo								■ ■ ■ ■ ■ ■ ■ ■

Figure 9.1 Full Project Gantt Chart (April – October 2026)

Milestones: PP1 - Progress Presentation 1 (May 2026) | PP2 - Progress Presentation 2 (August 2026) | Final Submission (October 2026). All official IT4010 RP deadlines are met within the above timeline.

10. REFERENCES LIST

- [1] E. A. Lopez-Rojas, A. Elmir, and S. Axelsson, "PaySim: A financial mobile money simulator for fraud detection," in *Proc. 28th Eur. Modelling Simulation Symp. (EMSS)*, Larnaca, Cyprus, Sep. 2016, pp. 249–255.
- [2] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, Mar. 2018.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. Int. Conf. Learn. Representations (ICLR)*, San Diego, CA, USA, 2015.
- [4] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, Venice, Italy, Oct. 2017, pp. 2980–2988.
- [5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [6] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, Long Beach, CA, USA, Dec. 2017, pp. 5998–6008.
- [7] P. Alumona *et al.*, "Fraud detection in financial transactions using machine learning: Insights from PaySim," *World J. Adv. Res. Rev.*, vol. 28, no. 3, pp. 382–392, 2025.
- [8] M. Weber *et al.*, "Scalable graph learning for anti-money laundering: A first look," arXiv:1812.00076, Dec. 2018.
- [9] A. B. Arrieta *et al.*, "Explainable Artificial Intelligence (XAI): Concepts, taxonomies, opportunities and challenges toward responsible AI," *Inf. Fusion*, vol. 58, pp. 82–115, Jun. 2020.

11. APPENDICES

Appendix A - Risk Analysis Register

Risk	Severity	Prevention	Fallback
RAM insufficient for tensor materialisation	High	tf.data streaming pipeline - windows generated on-the-fly from features.parquet	Build in 500K-row chunks; load sequentially
TCN underfits — low Recall	High	BinaryFocalCrossentropy($\gamma=2.0$); EarlyStopping on val_recall; ReduceLROnPlateau	Lower decision threshold to 0.2 via precision_recall_curve
TCN overfits — val_recall drops	Medium	Dropout(0.2) per block; Dropout(0.3) Dense; batch=256; restore_best_weights	Increase Dropout to 0.4; reduce to 3 TCN blocks
fraud_attention shape error	Medium	2-epoch dummy test on 50K samples before full training	Replace with GlobalMaxPooling1D as fallback
Colab session timeout	Medium	ModelCheckpoint every epoch; resume from best_tstcn.h5	Use Colab Pro for stable sessions
Thread-safety bug in deque buffer	Medium	threading.Lock wrapping all deque operations; unit test with concurrent requests	Process requests sequentially; remove concurrency
API integration failure with Member 4	Low	Agree JSON schema by Month 3; test with curl before integration	Expose as Python function; Member 4 imports directly

Appendix B - Preliminary Visualisation Mockups

Appendix B.1 - Investigator Dashboard UI Mockup

The TS-TCN Investigator Dashboard presents fraud alerts generated by the temporal detection component in a structured review interface. The top panel displays the flagged transaction details: account identifier (nameOrig), transaction type, amount, and the computed temporal_risk_score gauge rendered as a colour-coded indicator (green ≤ 0.3 , amber 0.3–0.7, red ≥ 0.7). Below this, the fraud_attention heatmap renders the 32-window attention weights as a horizontal bar sequence, with the peak attention position highlighted and annotated with the triggering predecessor's feature values (drain_ratio, post_transfer_ratio, dest_was_empty, type_risk).

The middle panel presents the evidence block in a two-column layout: the left column shows the current transaction's key signal values with visual severity indicators, and the right column displays the triggering predecessor's features alongside the predecessor_signal label. The bottom panel exposes the raw JSON response for technical investigators, enabling direct inspection of all returned fields including the composite_id cross-reference key used by Member 4's LLM forensic layer.

The dashboard is implemented as a Streamlit component consuming the FastAPI /api/v1/classify endpoint in real time. All visual elements update automatically on each incoming transaction. The

layout is designed to enable a trained fraud analyst to triage a flagged alert within 30 seconds without requiring access to underlying model internals.

Figure A1 - fraud_attention Heatmap with Peak Feature Annotation: a Seaborn heatmap of fraud_attention weights across all 32 window positions for a sample fraud transaction, annotated with the feature values (drain_ratio, origin_zero_flag, dest_was_empty, type_risk) of the peak attention transaction. This provides both which timestep the model attended (positional explainability) and why that timestep was suspicious (feature-level evidence at that position). This is the primary explainability artefact and the unique output that no teammate produces.

Figure A2 - The Four-Model ROC Curve Comparison is a single plot that displays four ROC curves: isFlaggedFraud with an AUC of approximately 0.501, Logistic Regression at roughly 0.92, MLP at about 0.95, and TS-TCN at around 0.98, with AUC values labelled on each curve. The progressive improvement from a simple rule through linear and non-linear models to a full sequential architecture provides empirical justification for the TCN design.

Figure A3 - Feature Separation Histogram: overlapping histogram of drain_ratio (F1) for fraud versus non-fraud transactions. Shows the near-perfect separation at drain_ratio → 1.0 that the TCN learns as its primary signal. Accessible to any audience without ML background.

Figure A4 - Ablation Study Results Chart: a grouped bar chart comparing F1, Recall, and AUC-ROC across all four ablation configurations (A1–A4). Shows the quantitative contribution of the fraud_attention layer and the empirical validation of W=32 as the optimal window size.

Appendix C - API Schema Specification

The following schema governs the request and response format agreed with Member 4 by Month 3. The response uses PaySim's nameOrig column as the shared cross-component reference key, the same field that Member 1 uses as graph node IDs and Member 2 uses as VAE account keys, enabling Member 4's LLM to cross-reference the flagged account across all three upstream models.

Request: POST /api/v1/classify

```
{ "nameOrig": "C1231006815", "step": 214, "type": "TRANSFER", "amount": 450000.0, "oldbalanceOrg": 451000.0, "newbalanceOrig": 0.0, "oldbalanceDest": 0.0, "newbalanceDest": 449000.0 }
```

Response:

```
{ "transaction_ref": { "nameOrig": "C1231006815", "step": 214, "composite_id": "C1231006815_214" }, "temporal_risk_score": 0.974, "risk_level": "CRITICAL", "detection_method": "TS-TCN", "modality": "temporal_sequence", "evidence": { "current_transaction": { "type": "TRANSFER", "amount": 450000.0, "drain_ratio": 0.999, "post_transfer_ratio": 0.001, "dest_was_empty": 1, "dest_enrichment": 0.998, "type_risk": 0.499, "hour_of_day": 0.609, "fraud_signal_summary": "Account fully drained into empty destination account" } }, "triggering_predecessor": { "nameOrig": "C1231006815", "step": 211, "composite_id": "C1231006815_211", "attention_weight": 0.847, "offset_from_current": -3, "features": { "type": "TRANSFER", "amount": 180000.0, "drain_ratio": 0.401, "post_transfer_ratio": 0.599, "dest_was_empty": 1, "type_risk": 0.499 }, "predecessor_signal": "Prior partial drain from same account — escalating fraud pattern" } }
```

Schema field guide:

`transaction_ref` - contains `nameOrig` (PaySim originator account ID), `step` (simulation hour), and `composite_id` (`nameOrig_step`, guaranteed unique). Member 4 uses `composite_id` as the primary case reference and `nameOrig` to query M1's graph and M2's VAE for the same account.

`temporal_risk_score` - float 0–1, the TS-TCN's fraud probability. Feeds directly into Member 4's Weighted Ensemble Meta Classifier alongside M1 and M2 scores.

`evidence.current_transaction` - all 10 computed features plus `fraud_signal_summary`, a one-line human-readable label for the LLM's Chain of Evidence prompt template. The LLM uses this as a factual anchor rather than generating descriptions from raw numbers.

`triggering_predecessor` - the most attended preceding transaction identified by `fraud_attention`. Contains `nameOrig` and `composite_id` (M4 can query M1/M2 for this account), `attention_weight` (the actual weight from the attention layer), `offset_from_current` (negative integer, e.g. -3 means 3 positions back in the window), its 6 key features, and `predecessor_signal` (human-readable label of the escalation pattern). This provides both which account preceded the fraud and the concrete feature values that made it suspicious.

`modality` - hardcoded 'temporal_sequence', telling Member 4 which detection channel this score represents within the DeepSentinel ensemble.

Appendix D - Ethical Considerations Statement

This research uses the PaySim mobile money simulation dataset, which is synthetically generated from real aggregate statistics and contains no personally identifiable information, no real financial account data, and no sensitive personal attributes. The dataset is publicly available on Kaggle under an open access licence and requires no institutional ethics committee approval.

The TS-TCN produces risk scores intended to support human investigator judgement, not replace it. All flagged transactions are reviewed by a qualified fraud analyst before any adverse action is taken. The `detection_method` field in every API response indicates whether the score was produced by the TS-TCN model, ensuring full transparency in automated decision-making. In any production deployment, the operator must comply with applicable AML regulations and Sri Lanka's Personal Data Protection Act No. 9 of 2022.